

AI Framework 使用说明

团队共享流程合约；每位开发者本地维护自己的项目记录、知识与执行历史。

1. 定位与目录

中央 `ai_framework` 仓库发布共享规则。目标代码仓库提交入口、共享项目上下文、版本锁定和启动器；实际框架与个人记录只保存在主工作树被外层 Git 忽略的 `agent_bootstrap/`。

主工作树（团队入口 + 一份个人治理区）

```
AGENTS.md
PROJECT-CONTEXT.md
ai-framework.lock
tools/ai-framework
agent_bootstrap/
  .git/                # 个人流程记录历史
  project/             # 本地 DP、Sprint、Task、验收记录
  photo_app_knowledge/ # 名称由 lock 固定
  agents/、governance/、templates/
```

linked worktree（仅代码与受 Git 管理的入口）

```
AGENTS.md、PROJECT-CONTEXT.md、ai-framework.lock、tools/
src/ ...
不创建 agent_bootstrap/
```

所有 linked worktree 通过 Git common dir 自动读取主工作树的同一份 Agent 工作区；无需复制、软链接或额外路径配置。

2. 新项目初始化

项目创建者先从 AI Framework 发布包取得 `seed/tools/ai-framework`，放到目标项目的 `tools/ai-framework` 并赋予可执行权限。

```
mkdir -p tools
# 将 seed/tools/ai-framework 放入 tools/ai-framework
chmod +x tools/ai-framework
```

在目标项目的主工作树根目录执行：

```
./tools/ai-framework init \
  --source git@bitbucket.org:cool-flutter/ai.git \
  --ref <release-tag> \
  --project-name photo_app
```

初始化会创建 `AGENTS.md`、`PROJECT-CONTEXT.md`、`ai-framework.lock`、`/agent_bootstrap/` 忽略规则，以及主工作树中带个人本地 Git 的唯一 `agent_bootstrap/`。lock 固定 `project_name` 和 `knowledge_dir`；本地 Git 使用 `main` 分支，并自动创建首次基线提交。

```
git add AGENTS.md PROJECT-CONTEXT.md ai-framework.lock tools/ai-framework
.gitignore
git commit -m "chore: initialize AI Framework"
```

`--project-name` 可省略，但项目初始化者应确认自动生成的名称。lock 提交后，其他成员沿用该名称，不根据自己的克隆目录重新命名知识库。

初始化要求目标目录已是 Git 仓库，且不存在 `AGENTS.md`、`ai-framework.lock` 和 `agent_bootstrap/`。已有旧版复制接入时，请使用迁移流程。

3. 日常同步、检查与升级

同步与状态

```
./tools/ai-framework sync
./tools/ai-framework status
./tools/ai-framework path
```

`sync` 读取 lock，校验完整 commit，并更新主工作树公共 Agent 工作区中的共享合约、角色、治理和模板。`path` 输出 workspace、project 与 knowledge 的实际绝对路径；三个命令均可从 linked

worktree 执行。

创建 linked worktree 后

```
test -f AGENTS.md
test -f PROJECT-CONTEXT.md
test -f ai-framework.lock
test -x tools/ai-framework
test ! -e agent_bootstrap
./tools/ai-framework status
./tools/ai-framework path
```

linked worktree 不承载 `agent_bootstrap/`。定位失败时回到主工作树修复公共 Agent 工作区，不得在 Task worktree 初始化或复制另一份治理目录。

既有 worktree 已有旧目录时先执行 `git ls-files -- agent_bootstrap`。若仍被分支跟踪，先更新到移除治理目录的新基线；若只是本机副本，先核对并保留其中独有记录。脚本不会自动删除或合并旧副本。

升级框架

1. 从中央仓库发布记录获取目标 tag 与完整 commit。
2. 修改 `ai-framework.lock` 的 `ref` 与 `commit`。
3. 按目标代码仓库的正常流程审查、提交 lock 变更。
4. 每位开发者执行 `./tools/ai-framework sync`。

私有仓库使用 SSH key、ssh-agent 或系统 Git 凭据管理器认证。不要在 lock、启动器或 Git URL 中保存密码和 token。

4. 旧版 cp 接入项目迁移

旧项目已有 `agent_bootstrap/`，或旧 lock 缺少 `project_name / knowledge_dir` 时，不要运行 `init`。先放入新的项目启动器，再执行只读检查：

```
./tools/ai-framework migrate \
  --source git@bitbucket.org:cool-flutter/ai.git \
  --ref <release-tag> \
  --project-name photo_app \
  --dry-run
```

项目启动器会自动把目标设置为当前项目根目录。如果第一行单独执行后提示 `--target is required`，说明 `tools/ai-framework` 放入的是中央执行脚本，而不是 `seed/tools/ai-framework` 项目启动器。

确认报告后，明确授权替换旧共享规则：

```
./tools/ai-framework migrate \
  --source git@bitbucket.org:cool-flutter/ai.git \
  --ref <release-tag> \
  --project-name photo_app \
  --apply \
  --replace-shared
```

内容	迁移处理
<code>tools/ai-framework</code>	更新为目标发布版本的项目启动器
<code>agents/PROJECT-CONTEXT.md</code>	复制到目标工程根目录并交由项目 Git 管理
<code>project/</code>	原样保留
标准旧 <code>knowledge/</code>	移到 lock 指定的项目名知识库；自定义目录需人工识别
旧共享合约、角色、治理、模板	仅在 <code>--replace-shared</code> 下替换
<code>agent_bootstrap/AGENTS.md</code>	归档到 <code>.migration-archive/</code>
自定义根 <code>AGENTS.md</code>	默认停止；人工合并，或明确加 <code>--replace-root-agent</code>

若目标代码仓库已跟踪 `agent_bootstrap/`，迁移会停止。旧共享文件是否有本地修改无法自动推断，因此 `--apply` 必须显式附加 `--replace-shared`。

5. 个人本地记录 Git

主工作树的 `agent_bootstrap/.git` 只记录个人的本地 DP、Sprint、Task 和项目名知识库，并由所有 linked worktree 公共读取。共享的 `PROJECT-CONTEXT.md` 由目标代码仓库管理；`init` 已创建个人记录首次基线提交。

```
cd agent_bootstrap
git status
git add project photo_app_knowledge .gitignore
git commit -m "docs: record local planning and evidence"
```

Task 中的 `Delivery commit` 始终指向目标代码仓库中的代码提交，不是这里的本地记录提交。

6. 工作流

本地 DP (LDP) 记录当前工作中的决策边界、约束与取舍。命中产品方向、架构、安全、数据模型、权限、外部 API 等决策边界时，必须先接受相关 LDP；普通局部实现不需要新增 LDP。关联的 Sprint 与 Task 必须遵守已接受的 LDP。

可选讨论 (Discussion)：候选方案和未决问题，不是执行授权

Sprint Draft：明确目标、范围、非目标；命中决策边界时已有 Accepted LDP

Sprint Ready：验收、依赖、必要 LDP 与 Gate 计划齐备，才可创建 Task

Task：Ready → In Progress → In Review → Done

独立验收；按需执行集成与硬件 / 目标环境 Gate

Sprint Review Passed 后，Sprint 才可 Completed

Discussion 和 Draft 均不得创建 Task 或开始实现。范围、架构或合同变化时，回到 Issue → 产品经理审查 → LDP → Sprint / Task 更新。

Gate

- **独立验收 Gate:** 每个 Task 必需；验收人不得与 Task Owner 相同。
- **集成验收 Gate:** 跨 Task、模块或服务时适用。
- **硬件 / 目标环境 Gate:** 依赖实体设备、真机、发布环境或目标环境时适用。

环境搭建与部署

根目录 `PROJECT-CONTEXT.md` 记录项目长期有效的环境 ID、平台位置、部署入口和保护规则；某一次部署的目标、执行边界、人工授权、attempt 和结果记录在对应 Task。涉及目标环境变化时，先通过 `./tools/ai-framework path` 定位公共 Agent 工作区，再读取其中的 `governance/deployment-governance.md`。

Environment Change Authorization

- ****Target and Change:**** 平台工程师填写，项目经理核对。
- ****Execution Boundary:**** 平台工程师填写，项目经理核对。
- ****Approval:**** 项目经理记录人工 Owner 的授权；未批准时为 Pending。

产品经理定义目标与验收

项目经理创建部署 Task

平台工程师补全目标和执行边界

人工 Owner 批准完整范围

平台工程师在授权内执行、诊断和有限重试

交付 In Review，进行独立验收与必要的目标环境 Gate

授权针对有边界的环境变化，不针对每条命令。同一范围内的准备、诊断、修复和预算内重试保留在原 Task；Gate 验证结果，不作为每条环境操作的执行许可。

7. 四个 Agent 角色

角色	主要职责	边界
产品经理 Agent	分析方案，创建和维护本地 DP、Sprint	不写生产代码，不创建或调度 Task
项目经理 Agent	将 Ready Sprint 与其必要关联的 Accepted 本地 DP 转化为 Task，协调验收、Gate 和 Review	不执行部署，不代替人工 Owner 授权
高级工程师 Agent	按 Task 实现、测试、提交 delivery commit，并交付 In Review	不自行宣布 Task Done 或 Sprint 完成
平台工程师 Agent	按部署 Task 和环境变更授权搭建环境、交付配置与产物、部署、恢复并验证目标环境	不扩大授权范围，不验收自己的 Task